Module 11: Apex Classes

Module Objectives

At the end of this course, you will be able to:

- Describe the features of the Apex programming language
- Create and use specific and generic sObject variables
- Understand static variables and methods, apex properties, and with or without sharing
- Learn various OOPS concepts like inheritance, polymorphism and interface

```
Apex:
1. is a strongly typed language --> Data types are available and the variables are created with data type defined.
```

```
integer number = 10;
string name = 'Salesforce';

dynamically typed language --> python, javascript
        var number=10;
        var name='Salesforce';
```

What is Apex?

- **Apex** is a **strongly typed, object-oriented programming** language specifically designed for the **Salesforce** platform.
- Apex allows developers to execute flow and transaction control statements on Salesforce servers.
- It looks similar to Java and acts like database stored procedures.
- Developers can add business logic to various system events like related record updates and many more.

Features of Apex

- ✓ It is object oriented
- ✓ It is hosted - since it is over the cloud
- ✓ It is not case sensitive
- ✓ Easy to use
- ✓ Automatically Upgradeable
- ✓ Versioned
- ✓ Integration with DML calls, API's
- ✓ Multitenant

```
integer number = 10;
string name = 'Salesforce';

dynamically typed language --> python, javascript
        var number=10;
        var name='Salesforce';
2. is a object oriented programming language
3. Apex is hosted, means it is on cloud.
4. It is multitenant.
5. It is not case sensitive
6. It is versioned.(Using Api version)
```

Apex Programming vs. Other Object-oriented Programming Languages

Aspect	Apex Programming (Salesforce)	Other Object-Oriented Languages
Language Origin	Developed specifically for Salesforce, based on Java.	Developed independently (ex. Java, C#, Python, Ruby).
Platform Integration	Tightly integrated with Salesforce ecosystem.	Not tied to any specific platform.
Execution Environment	Runs on Salesforce servers (multi tenant).	Executed in various environments (local, cloud, etc.).
Syntax and Semantics	Similar to Java, easy to understand.	Syntax varies across languages.
Data Manipulation	Supports DML (INSERT, UPDATE, DELETE) and SOQL queries.	Varies based on language and database.
Type System	Strongly typed, direct references to schema objects.	Varies (strongly typed or dynamically typed).

Apex Class

- An Apex class serves as a template or blueprint from which Apex objects are created.
- It defines the behaviour and logic associated with an object or a specific functionality.

```
public class ClassName
{
    public void MethodName()
    {

    }
}
```

Access Specifiers

Access specifier	Accessibility
Public	This keyword is used to define a class, method, or variable that can be accessed by all the Apex code within a specific package.
Private	This keyword is used to define a class, method, or variable that is only known locally, within the section of code in which it is defined. This is the default access modifier.
Protected	This keyword defines a method/variable that is visible to the classes that extend the base class.
Global	This keyword is used to define the class/method/variable is visible to all code in any namespace.

```
Access Specifiers:
Public : it can be accessed through out the org.
private: it can be accessed within the class it is defined.
protected: it can be accessed within the class in which it is defined and also
inside the inherited class.
global: it can be accessed outside the salesforce org.(used in integration)
```

Apex Datatypes

Category	Data Type	Description
Primitive	Integer	Represents whole numbers (ex: 54)
	Double	Represents floating-point numbers (ex: 3.14)
	Long	Represents large whole numbers
	Date	Represents a date without a time component
	Datetime	Represents a date and time
	String	Represents text or character data
	ID	Represents a unique identifier (usually for records in Salesforce)
	Boolean	Represents true or false values
sObject (Salesforce Object)	sObject	An sObject represents a record in Salesforce. It's used to interact with database records.

Comments

A comment is a text remark added to the program with the purpose of explaining details about the source code.

Single-line comment

```
public class CommentsProgram
{
    public void singleLineComment()
    {
        // System.debug('Hello');
    }
}
```

Multi-line comments

```
public class CommentsProgram
{
   public void multiLineComments()
    {
        /*
         System.debug('Hey');
         System.debug('Bye');
         System.debug('Hi');
        */
    }
}
```

Keywords in Apex

Keywords	Description
static	Defines a method or variable that is associated with the class itself (not an instance)
final	Indicates that a method or variable cannot be overridden or modified
super	Refers to the parent class
this	Refers to the current instance of the class
new	Creates a new instance of a class
return	Used to return a value from a method
instanceof	Used to check if an object is an instance of a specific class
transient	Indicates that a variable should not be serialized
with sharing	Specifies that sharing rules are enforced for the class
without sharing	Disables sharing rules enforcement

Demo

11.1 Get started with Apex : Apex Datatypes

Problem Statement
Let's create a simple Apex class that calculates the area of a rectangle. We'll assume that the rectangle's length and width are provided as input parameters.

Requirements
- Apex class name: RectangleAreaCalculator
- Method Name: calculateArea()
- Parameters: length and width

```
Any Application, has 3 layers:

1. Frontend --> User interface--> View --> View will be created with LWC
components.
2. Logic --> Controller --> will be created with Apex class
3. Backend --> Data Model --> Model --> Objects and fields --> already created

Data model interacts with the database.
```

```
Array: Array stores the list of elements (integers, dates, string etc)
Arrays are static means we define the size of array at compile time.

integer arr[] = new integer[10];

How collections are different from Array?
-->
```

Collections in Apex

Collections are fundamental data structures that allow you to organize and manage multiple values together.

- It is dynamic and growable in nature.
- It has some predefined functions.

In Apex, there are three main types of collections:

1. **List**
   - A list is an ordered collection of elements, each distinguished by its index.
   - *Syntax: List<datatype> name = new List<datatype>();*
   - Example: List<String> fruitList = new List<String>{'Apples', 'Bananas', 'Cherry'};

11.1 Get started with Apex

Collections in Apex (Continued)

List Functions	Purpose
add(element)	Adds an element to the end of the list
add(index, element)	Inserts an element into the list at the specified index position
set(index, element)	Sets the specified value for the element at the given index.
remove(index)	Removes the list element stored at the specified index, returning the element that was removed.
clear()	Removes all elements from a list, consequently setting the list's length to zero.
get(index)	Returns the list element stored at the specified index.
addAll()	Adds all the elements in the specified list to the list that calls the method. Both lists must be of the same type.
clone()	Makes a duplicate copy of a list.
deepClone(preserveId, preserveReadonlyTimestamps, preserveAutonumber)	Makes a duplicate copy of a list of sObject records, including the sObject records themselves.

>


```
list<integer> numbers = new list<integer>();

Difference between list and set:
1. list is an ordered collection whereas set is an unordered collection.
2. list allows duplicate values whereas set don't allow duplicate values.
3. list can be used with DML whereas set cannot be used with DML.
```

- Index starting from 0.

2. Set

- A set is an unordered collection of elements that does not allow duplicates.
- *Syntax: Set<datatype> name = new Set<datatype>();*
- Example: Set<Integer> uniqueNumbers = new Set<Integer>{1, 2, 3, 4};

Set Functions	Purpose
add(setElement)	Adds an element to the set if it is not already present. If the element is already in the set, it remains unchanged.
addAll(fromList)	Adds all elements from a specified list to the set if they are not already present.
addAll(fromSet)	Adds all elements from another set to the current set if they are not already present. Helps combine two sets while ensuring uniqueness.

3. Map

- In a map, the data is stored in the key-value pair; keys are always unique, and values can be duplicated.
- *Syntax: Map<datatype,datatype> name = new Map<datatype,datatype>();*
- Example: Map<String, Decimal> productPrices = new Map<String, Decimal>

```
{
    'Apple' => 1.99,
    'Banana' => 0.79,
    'Chocolate' => 2.49
};
```

```
public void MapExample()
{
    map<integer, string> employeeData = new map<integer,string>();
    employeeData.put(128963, 'Rohit');
    employeeData.put(318963, 'Anjali');
    employeeData.put(784633, 'Amit');
    employeeData.put(543323, 'Isha');
    employeeData.put(342543, 'Manish');
}
```

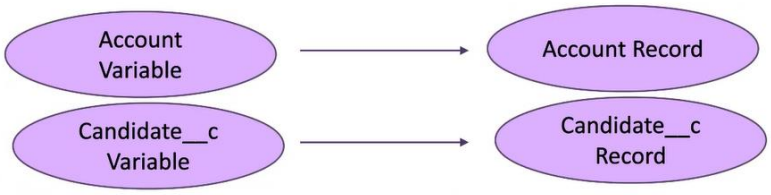
Map Functions	Purpose
put(key, value)	Adds or updates an entry in the map
get(key)	Retrieves the value associated with a given key
containsKey(key)	Checks if a key exists in the map
remove(key)	Removes an entry from the map
keySet()	Returns a set of all keys in the map
values()	Returns a list that includes all the values stored in the map.

```
set<integer> keyData = employeeData.keySet();
system.debug(keyData);

list<string> valueData = employeeData.Values();
system.debug(valueData);
```

```
What is sObject?
--> It stands for Salesforce object.
--> It is a datatype which is used to store any salesforce object record.

integer number;
```

Specific Object

Variables that are declared with the specific sObject data type can reference only the Salesforce records of the same type.



Examples:

1. Account acct = new Account(Name='Burlin', Phone='(41)54-1212', Industry='Banking');
2. Candidate__c cd = new Candidate__c(First_Name__c='Bob', Last_Name__c='Patrick');

Static Variables & Methods

Non-Static Variables and Methods

- Non-static methods and non-static variables are associated with specific instances (objects) of a class.
- They have no definition modifier (no static keyword).

```
public class StaticClass
{
   public String name = 'Mayur';
}
```

```
public class StaticClass
{
    public void nonStaticMethod()
    {
       System.debug('Without Static');
    }
}
```

11.4 Apex Properties

Introduction

An apex property provides a way to encapsulate data within a class, allowing you to customize the behavior when reading or modifying its value.

Property definitions include two code blocks:

1. Get accessor: Executes when the property is read.
2. Set accessor: Executes when the property is assigned a new value.

```
Public class ApexPropertiesClass
{
  // Property declaration
  access_modifier return_type property_name
  {
      get {
        //Get accessor code block
      }
      set {
        //Set accessor code block
      }
   }
}
```

Using Automatic Properties

Automatic properties provide a concise way to define properties without explicitly writing additional code in the get and set accessor code blocks.

Example:

```
public class ApexAutomaticProperty
{
   public integer studentId { get; }
   public double studentFees { get; set; }
   public string studentName { set; }
}
```

Demo

11.4 Apex Properties

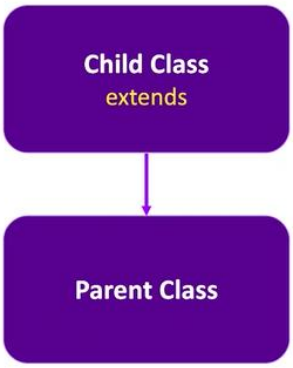
Problem Statement
You are building an employee management system in Salesforce using Apex. Your task is to create an Employee class that encapsulates employee data. The class should have the following properties and methods:

Properties:
- empName: The full name of the employee.
- empAge: The age of the employee.

Methods:
- setEmpData: A method that allows setting the employee's name and age.
- getEmpName: A method that retrieves the employee's full name.
- getEmpAge: A method that retrieves the employee's age.

What is Inheritance?

- Inheritance allows one class (the child class) to acquire properties and behaviors from another class (the parent class).
- When you create a new class by inheriting from an existing class, you can reuse methods and fields from the parent class.
- The child class extends the parent class using the **extends keyword** in its class definition.



Keywords Used in Inheritance

- **extends :** Defines a class that extends another class.
- **virtual :** This keyword Defines a class or method that allows extension and overrides.
- **final :** All apex classes are by default final and should be explicitly mentioned as virtual or abstract to participate in inheritance

Demo

11.5 Extending a Class

Problem Statement
Suppose we are building an application to manage different types of Vehicles. We want to create a base class called **Vehicle** with common properties like **brandName** and method as a **vehicleType**. Then, we'll create a derived class called **Car** that inherits from Vehicle and adds specific properties like **modelName**.

Requirements:
Implement a method called vehicalType(String vehicleCategory) that represents the type of vehicle (ex. "Car", "Bike", or "Auto").
Conditions:

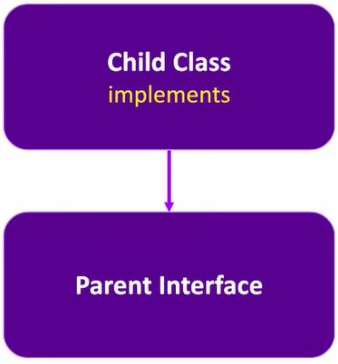
- If vehicleCategory is "Car" print "Vehicle Type: 4 Wheeler".
- If vehicleCategory is "Bike" print "Vehicle Type: 2 Wheeler".
- If vehicleCategory is "Auto" print "Vehicle Type: 3 Wheeler".
- Otherwise, print "Invalid Vehicle Type".

```
Method Overriding --> can be done where we have implemented inheritance.

class A
{
public void methodA()
{

}
}
class B extends A
{
public void methodA()
{

}
```

11.7 Interface

Introduction

- An interface is like a blueprint or a contract for classes.
- It defines a set of method signatures (without implementations) that other classes must adhere to.
- Interfaces provide a layer of abstraction, separating method declarations from their specific implementations.
- When a class implements an interface, it must provide concrete implementations for all the methods defined in that interface.



11.7 Interface

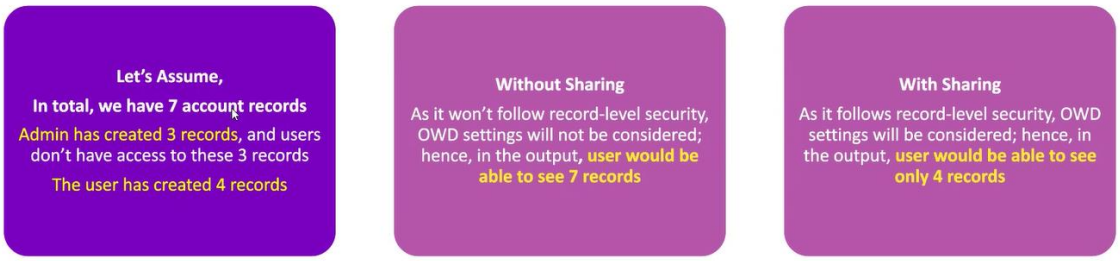
Keywords Used in Interface

- **Interface :** This keyword is used to define a data type with method signatures. Classes implement interfaces. An interface can extend another interface.
- **implements :** This keyword is used to declare a class that implements an interface.

11.8 With or Without Sharing

Introduction

1. **With Sharing:** For class, the sharing settings(profile permissions, FLS, OWD, Sharing rules) will be enforced.
2. **Without Sharing:** The sharing settings will not be enforced. This is by Default setting.



Demo

Without Sharing

Apex Class: To print total no of account records

```
public without sharing class WithoutSharingProgram
{
    public static void displayAccountCount()
    {
        // Let's assume Account object has total 7 records,
        // Admin has inserted 3 & user has inserted 4 records
        List<Account> lst = [Select Id From Account];
        System.debug('Count is'+lst.size());   // total no of account records
    }
}
```

Output: 7

Demo

With Sharing

Apex Class: To print total no of account records

```
public with sharing class WithoutSharingProgram
{
    public static void displayAccountCount()
    {
        // Let's assume Account object has total 7 records,
        // Admin has inserted 3 & user has inserted 4 records
        List<Account> lst = [Select Id From Account];
        System.debug('Count is'+lst.size());   // total no of account records
    }
}
```

Output: 4

With Sharing --> Sharing settings(OWD, Sharing Rules, Manual Sharing, Role Hierarchy) will be enforced inside the apex class.
OWD --> Organization Wide Defaults (discussed in Module 4 Data Security)

Without Sharing --> Sharing setting will not be enforced inside the Apex class.